Ron, let’s connect this to what you already know 🔁

In Python, you might loop over a list like this:

Python
```
for color in ["red", "green", "blue"]:
    print(color)
```
In JavaScript, we can do something very similar, but we let the array call a function for each item using forEach().

Very roughly, it looks like:

Javascript
```
colors.forEach(/* a function goes here */);
```
You already know functions can be passed around as values — forEach() is just “use this function on every item in the array”.

Quick check before we touch code:

If I say: “forEach() will run my function once for each item in the array, in order” — does that idea feel clear enough to start, or a bit fuzzy?

Just answer:

“clear”
or
“fuzzy, explain with an example”
Clear

Nice 👌 That’s the core idea.

Let’s make it concrete with a tiny array.

Here’s the shape we’ll use:

Javascript
```
const colors = ["red", "green", "blue"];

colors.forEach(function (color) {
  console.log(color);
});
```
Step by step, what happens:

colors is an array of 3 strings.
forEach(...) goes through the array from left to right.
For each item, it calls the function you gave it.
That function gets one parameter here: color → the current item.
Inside, we console.log(color) once per item.

So the function is called 3 times: once with "red", then "green", then "blue".

Before you touch the editor:

If you ran that code, what do you expect to see in the console, line by line? (Just describe it in words, no need to be fancy.)

red
geen
blue

Perfect — and I see that little typo: you wrote geen instead of green 😉

So the console would show:

red
green
blue

One per line, in that order, because forEach() walks the array from index 0 to the end and calls your function each time.

Next tiny step 💞

Now I want you to run a version of this yourself.

In the code editor, do this:

Make an array of 3–4 of your favorite foods, like:

const foods = ["pizza", "tacos", "sushi"];
Use forEach() with an inline function to log each food:

```
foods.forEach(function (food) {
  console.log(food);
});
```

Hit Run first, see the output.

When it looks good, hit Submit code so I can see it.

Don’t paste code here — write and run it in the editor. After you submit, I’ll help you look at what happened step by step.

done

Nice work getting that done 👏

Let’s zoom in on one key idea now: the value that forEach() gives your function each time.

In your pattern:

Javascript
```
foods.forEach(function (food) {
  console.log(food);
});
```

foods is the array.
forEach(...) goes through it.
Each time, it calls your function and fills food with the current item in the array.

So if foods is:

Javascript
```
["pizza", "tacos", "sushi"]
```

then your function runs 3 times with:

food === "pizza"
food === "tacos"
food === "sushi"
Quick check, just to be sure:

If you changed the parameter name from food to item:

Javascript
```
foods.forEach(function (item) {
  console.log(item);
});
```
do you think it would still work the same?

Answer “yes” or “no”, and then I’ll explain why.

yes

Yes, exactly ☑

The name food or item is just a label you choose. What matters is:

The first parameter of the forEach callback always receives the current array value.

So in both cases:

Javascript
```
foods.forEach(function (food) { ... });
foods.forEach(function (item) { ... });
```
that first parameter is: "pizza", then "tacos", then "sushi" as the loop runs.

Next step: there’s a second parameter we can use: the index (position in the array).

I’ll explain that one next. Ready to see how index works?

Yes, show me

Awesome, let’s keep it simple and visual 🍸

So far, your function got one thing each time: the value (food, item, etc).

forEach can also give you a second thing: the index — the position in the array (starting at 0).

Basic shape:

Javascript
```
array.forEach(function (item, index) {
  console.log(item, index);
});
```
item → current value
index → position: 0, 1, 2, ...

Example with colors:

Javascript
```
const colors = ["red", "green", "blue"];

colors.forEach(function (color, index) {
  console.log(index, color);
});
```
This would log:

0 "red"
1 "green"

2 "blue"
Make sense so far:

item = value, index = position starting at 0?

Just answer “yes” or “a bit confused”.

Yes, that makes sense

Great ☑ Let’s use index in your own code now.

Here’s the pattern again, with both parameters:

Javascript
```
foods.forEach(function (food, index) {
  console.log(index, food);
});
```
Step by step:

On the 1st item → index is 0
2nd item → index is 1
3rd item → index is 2
and so on…
Your turn in the code editor 🖉

Keep your foods array from before (or make a new one).
Change your forEach so the callback has two parameters and logs both, like above.
Hit Run to see the output.
Then hit Submit code so I can review it.

Do that now, and I’ll help you read the output line by line.

Javascript
```
const foods = ["scallops", "steak", "crabs"];


foods.forEach(function (food, index) {
    console.log(index, food);
});
```
Love this example, Ron — and nice choice of foods 😄

Your code:

Javascript
```
const foods = ["scallops", "steak", "crabs"];

foods.forEach(function (food, index) {
    console.log(index, food);
});
```
Output:

0 scallops
1 steak
2 crabs

That shows you clearly get:

food → current value
index → current position (0, 1, 2)
Next key idea for this lesson:

What does forEach() itself return?

If you did this:

Javascript
```
const result = foods.forEach(function (food, index) {
  console.log(index, food);
});

console.log("Result is:", result);
```
what do you think would be logged for result?

You can just guess in words:

“an array”,
“the last item”,
“undefined”,
or something else.
undefined we didnt pass anything out that I see

Exactly right: undefined ☑

You spotted the key idea: inside the callback you log things, but you don’t return anything from forEach(). So:

Javascript
```
const result = foods.forEach(function (food, index) {
  console.log(index, food);
});

console.log(result); // this will be undefined
```
The side effect: logs each food and index.
The return value of forEach() itself: undefined.

So forEach() is for doing something for each item, not for building a new array.

Last core concept: translating this to a common pattern.

Think in Python:

Python
```
total = 0
for score in [10, 20, 30]:
    total = total + score
```
We can do the same with forEach() in JavaScript: use a variable outside, update it inside the callback.

In the code editor, try this:

Make an array of numbers, like scores:

```
const scores = [10, 20, 30];
```
Make a let total = 0; before the forEach.

Use forEach() with (score, index) and inside do: total = total + score;

After the forEach, do console.log("Total is:", total);

Run, then Submit code.

Once you submit, we’ll check:

callback runs once per item,
item vs index,
and the final total.
Javascript
/*
const foods = ["scallops", "steak", "crabs"];

foods.forEach(function (food, index) {
    console.log(index, food);
});
*/
const scores = [5, 15, 20, 30];
let total = 0;
scores.forEach(function (score, index) {
    total = total + score;
});
console.log("Total is:", total);
That’s beautifully done, Ron 🎯

Your code:

Javascript
const scores = [5, 15, 20, 30];
let total = 0;

scores.forEach(function (score, index) {
    total = total + score;
});

console.log("Total is:", total);
Step by step:

scores has 4 numbers.
forEach calls your function 4 times, once per score.
Each time, score is the current value, index is the position (0–3).
total accumulates: 0 → 5 → 20 → 40 → 70.
Final log: Total is: 70 ☑

You’ve now shown you can:

Use forEach() with a callback
Use item and index correctly
Understand that forEach() is for side effects, not return values
One last small twist, then we’re done with this concept:

Right now your callback is a normal function.

We can also write it as an arrow function:

Javascript
scores.forEach((score, index) => {
  total = total + score;
});
Same behavior, just shorter syntax.

Can you go into the editor, change your callback to an arrow function like that, run it, and Submit code again?

I just want to see you use forEach + arrow style once.

Javascript
/*
const foods = ["scallops", "steak", "crabs"];

```
foods.forEach(function (food, index) {
    console.log(index, food);
});
*/
const scores = [5, 15, 20, 30];
let total = 0;
//scores.forEach(function (score, index) {
//  keeping the old for my notes
scores.forEach((score, index) => {
    total = total + score;
});
console.log("Total is:", total);
That’s perfect, Ron 🎉


Your final version:


const scores = [5, 15, 20, 30];
let total = 0;

scores.forEach((score, index) => {
    total = total + score;
});

console.log("Total is:", total);

You’ve shown you can:


Use forEach() to run a callback for each array item
Use item and index correctly inside the callback
Understand that forEach() does work (logs, updates total) but returns undefined itself

You’ve fully nailed this lesson ☑




Quick recap in plain words:


array.forEach(callback) → runs callback once per item, in order.
First parameter (item) = the value.
Second parameter (index) = the position (0, 1, 2, …).
It’s great for “do something with each item”, not for “give me a new array back”.


If you want, you can experiment a bit more on your own:


Try logging both index and score in the arrow version.
Or use forEach on a list of strings (like your foods) with an arrow callback.

When you’re ready, you can jump into the next lesson in Learn. I’ll be right here for
the next step 🚀

/*
const foods = ["scallops", "steak", "crabs"];


foods.forEach(function (food, index) {
    console.log(index, food);
});
*/
const scores = [5, 15, 20, 30];
let total = 0;
//scores.forEach(function (score, index) {
```

```
472   //  keeping the old for my notes
473   scores.forEach((score, index) => {
474       total = total + score;
475   });
476   console.log("Total is:", total);
477
```